Project Plan Document

Course: BIL 481 — Software Engineering
Project: Model Context Protocol (MCP) Based Campus Data Orchestrator
Official Short Title: Campus Data Orchestrator
Team: Group-ASYM
Document status: Version 1.0 (Final Deliverable — reviewed and verified for submission)
Initial assignment due date: 1 November 2026, 23:59 (Türkiye time)
Planned project closure: 15 January 2027

Project team: Murat Can Işık (241101013), Yunus Emre Biçer (241101009), Ahmet Sefa Efendioğlu (241101066), Arda Çelik (241101065).

> **Document authorship: Yunus Emre Biçer** (Lead Author, document preparation), **Ahmet Sefa Efendioğlu** (QA Reviewer, cross-document audit, submission coordination). Document-writing assignments are separate from later software implementation roles.

Table of Contents

1. Project Scope
2. Project Organization and Responsibilities
3. Project Execution Objectives
4. Key Phases, Timeline, and Milestones
5. Resource Planning
6. Risk Management
7. Communication Plan
8. Change Management Plan
9. Budget Plan
10. Effort Estimation
11. Task Assignment Matrix and Rationale
12. Document-Specific Task Matrix

1. Project Scope

Campus Data Orchestrator aims to provide a unified interface for selected campus information sources. The planned system exposes data through a Model Context Protocol (MCP) server and allows students to obtain straightforward information through direct dashboard widgets or combine multiple sources through an LLM-assisted conversational interface.

In scope:

- A TypeScript/Node.js MCP server with defined tools for cafeteria menus, library occupancy, shuttle schedules, and course schedules.
- A web-based interface with a dashboard and conversational view.
- A deterministic intent router that directs simple, single-service requests to a direct data path and compound requests to the LLM agent.
- An API gateway providing authentication verification and separation of user credentials from LLM prompts.
- Redis-backed caching, stale-if-error responses, and failure isolation for upstream data sources.
- Modular campus connectors accompanied by synthetic/mock implementations for development and testing.
- Automated tests and deployment/documentation artifacts suitable for a project demonstration.

Out of scope:

- Modification of university records, course registrations, payments, or other write operations in campus systems.
- Replacement of official university portals or ownership of their underlying data.
- A guaranteed production-grade launch or unrestricted access to private university APIs.
- Features outside the four campus domains identified in the Concept Notes.

External dependency: Actual integration with university services depends on authorized access and available endpoints. The previously submitted Concept Notes explicitly include synthetic/mock connector modes; these will allow development, testing, and a demonstration without unauthorized access. If live integrations or university SSO cannot be implemented, the team will document the limitation and use the change-management process rather than claim simulated behavior is a verified live integration.

2. Project Organization and Responsibilities

The following implementation allocation is the team's current **planning proposal**, accepted for this draft by the Project Plan author; all four members should acknowledge their assignments before coding starts. These roles concern the *software project*, not Assignment 1 document authorship.

Team member	Main implementation responsibility	Primary deliverables
Murat Can Işık	MCP server and LLM integration	MCP tool registry, protocol/dispatcher, agent tool orchestration, Gemini integration
Yunus Emre Biçer	Frontend and deterministic routing	Next.js dashboard, conversational UI and streaming feedback, bilingual intent router
Arda Çelik	Campus connectors and resilient data access	Four campus data connectors, deterministic synthetic fixtures, Redis caching, stale-if-error and circuit breaking
Ahmet Sefa Efendioğlu	API gateway, authentication, security, integration QA	JWT/mock-SSO gateway, rate limiting and credential isolation, cross-module integration tests, release checks

All members will test their own modules and contribute to integration. The QA lead coordinates final verification rather than performing every test alone. Workload aims for **17.5 estimated person-hours per member**, recognizing that task count does not equal task difficulty; the group will redistribute tasks if actual effort becomes uneven.

Assignment 1 document ownership: Arda — Project Definition Document; Murat — Requirements Document; Yunus — Project Plan Document; Sefa — cross-document review, PDF conversion, and submission coordination. These ownership assignments do not determine who implements each technical feature.

3. Project Execution Objectives

1. Keep the submitted Concept Notes as the initial scope baseline and use the Requirements Document's **FR-01–FR-20** and **NFR-01–NFR-09** as the detailed verification reference.
2. Implement four MCP-exposed campus services (cafeteria, shuttle, library occupancy and course schedules), using reproducible mock data when official campus sources are inaccessible.
3. Demonstrate the client-side direct path for atomic queries and the Gemini-assisted MCP tool orchestration path for compound queries, when the free API quota permits a live demonstration.
4. Maintain the gateway's authentication boundary, protect credentials and personal information, and mark mock or stale results accurately.
5. Integrate work through identifiable GitHub branches and peer-reviewed pull requests; track blockers, decisions and actual effort.
6. Run module tests, integration tests and requirements-based acceptance checks; identify completed, partially implemented and unimplemented requirements in the project evaluation.

Implementation order (not a reduction of formal requirements): First establish the MCP server, the four synthetic campus sources and a basic web interface; then connect Gemini and verify multi-tool queries; then integrate cache, gateway, reliability and security behavior; finally run broader performance, usability and deployment checks. Some stages may overlap. The Requirements Document's *Must / Should / Could* levels remain authoritative; this order does not silently downgrade a Must requirement. The course team is understood not to require 100% implementation, so any incomplete features will be explicitly documented rather than presented as completed.

4. Key Phases, Timeline, and Milestones

All dates below follow the team's Assignment 1 task-allocation sheet. Dates other than the **1 November 2026 Assignment 1 deadline** are *team planning targets*, not additional official deadlines announced in the course assignment.

Phase	Planned dates	Estimated effort	Main outputs and internal milestones
1. Analysis and planning	19 Oct – 1 Nov 2026	10 person-hours	Project Definition, Requirements, Project Plan; Assignment 1 due **1 Nov**
2. Design and skeleton	2 – 20 Nov 2026	12 person-hours	Architecture, initial interfaces, component skeletons, design review by **20 Nov**

Phase	Planned dates	Estimated effort	Main outputs and internal milestones
3. Core development	21 Nov – 25 Dec 2026	36 person-hours	MCP server, UI, connectors, gateway and cache integrations; integration checkpoint by **25 Dec**
4. Testing and delivery	26 Dec 2026 – 15 Jan 2027	12 person-hours	Test results, fixes, demonstration, documentation, closure and evaluation by **15 Jan**
Total	**19 Oct 2026 – 15 Jan 2027**	**70 person-hours**	

Simple Gantt chart (approximate):

Phase	Oct 2026	Nov 2026	Dec 2026	Jan 2027
Analysis and planning	▇▇▇	▏		
Design and skeleton		▇▇▇		
Core development		▇	▇▇▇	
Testing and delivery			▇	▇▇

Planned handoffs: Requirements and scope are baselined at the end of Phase 1; APIs, schemas, and component boundaries are agreed by the end of Phase 2; integrated core features are demonstrated by the end of Phase 3; testing evidence and final deliverables are checked in Phase 4. The course's later assignment dates have not been specified in the supplied Assignment 1 instructions and must be added if announced.

Closure and evaluation: On 15 January 2027, the team will review completed requirements, outstanding limitations, test evidence, and lessons learned, then archive or tag the demonstrated project state. This is a proposed internal evaluation date.

5. Resource Planning

Resource category	Planned resources	Access and purpose
Team	Four Group-ASYM members	Design, implementation, peer review and testing
Collaboration	WhatsApp; the existing GitHub repository, Issues and pull requests	Quick coordination and traceable decisions; no new paid collaboration tools
Branch strategy	Stable `main`, integration `dev`, separate named feature branches	After review, merge feature PRs into `dev`; promote tested changes to `main`. Existing Assignment 1 documentation PRs need not be reworked solely to introduce `dev`.
Backend and MCP	Node.js, TypeScript, Fastify, MCP SDK	MCP protocol/tools and API gateway
Frontend	React, Next.js, Tailwind CSS	Dashboard, chat, route selection and streaming UI

Resource category	Planned resources	Access and purpose
Data and infrastructure	Redis, PostgreSQL/Prisma, Docker Compose	Caching, audit records, local multi-container integration
LLM	Google Gemini API (free allowance, if available); local mock responses for tests	Multi-tool natural-language response synthesis; no paid API usage is planned
Testing	Jest, Supertest, mock fixtures; load-testing utilities if feasible	Unit tests, API/integration tests and quality measurements
University services	Only authorized endpoints, plus deterministic mock connectors	Four campus data domains; live endpoints and SSO are not assumed available
Hardware	Members' existing computers	Local development and demo; no public deployment planned

External API credentials must be held in untracked environment configuration, never committed to the repository. Actual free-tier eligibility and limits will be verified at integration time; a mocked AI response tests the application path but does **not** by itself prove the live LLM requirement.

6. Risk Management

The risk register below aligns directly with the Risk IDs (R-1 to R-10) defined in Section 9 of the Project Definition Document, mapping each risk to its operational lead and mitigation strategy:

| ID | Risk | Likelihood / Impact | Mitigation and fallback | Responsible lead | |---|---|:---:|---|---| | **R-1** | No authorized campus SIS, occupancy or transport API access | High / Medium | Use synthetic connectors explicitly supported by the Concept Notes; label simulated data and record any live integration limitations | Arda / Sefa | | **R-2** | External service outage, cache expiry, or slow response | Medium / High | Redis caching, stale markers, 800 ms upstream timeout, isolation and circuit breakers, fault-injection tests | Arda / Sefa | | **R-3** | Free Gemini API quota/key unavailable or rate-limited | Medium / Medium | Verify free availability before integration; intent router bypass for simple queries; mock LLM outputs for offline tests | Murat | | **R-4** | LLM gives wrong or invented answers (hallucination) | Medium / High | Agent answers strictly from tool outputs; low temperature setting; tool telemetry badges; scripted test suite | Murat | | **R-5** | Unintended exposure of tokens, secrets, or PII to LLM | Low / High | Gateway perimeter token termination; PII scrubbing; audit tests to verify zero credential leakage | Sefa / Murat | | **R-6** | Scope exceeds the initial 70 person-hour estimate | High / High | MoSCoW prioritization; drop optional items first; track weekly hours; scope review at phase gates | Yunus / All members | | **R-7** | Team member unavailable (illness, exams, course load) | Medium | Medium | Secondary code reviewers for each module; transparent task board tracking on GitHub Projects | All members | | **R-8** | Unfamiliar technologies (MCP SDK, SSE, Redis) slow progress | Medium / Medium | Early technical spike in Phase 2; small vertical prototype (one tool end-to-end) before full implementation | All members | | **R-9** | Independently developed modules fail to integrate | Medium / Medium | Freeze tool schemas and JSON contracts in Phase 2; small, frequent PRs into `dev`; CI automated tests | Sefa / All members | | **R-10** | Merge conflicts or lost work in GitHub | Low / Medium | Strictly branch-based workflow; mandatory PR reviews before merging; continuous local rebasing | Sefa / All members |

Risk scores are provisional judgments, not observed failure frequencies. Owners will review the register during weekly coordination and after any significant change.

7. Communication Plan

- **Routine communication:** WhatsApp for questions, blockers and brief coordination; a short **weekly written status check** (completed work, next tasks, blockers) rather than mandatory long meetings.
- **Traceability:** GitHub Issues document implementation tasks and important decisions; pull requests carry code review and change history. Each task has a primary owner and an expected outcome.
- **Version control:** Each person works on a named feature branch. For software development, open PRs into `dev` and obtain a teammate's review before merging; when the integrated version is tested, open a PR from `dev` into stable `main`. Assignment 1 documentation PRs already underway may follow the current repository workflow.
- **Shared interfaces:** Affected members agree tool schemas and data contracts in Phase 2; a modification to another person's interface must be discussed before integration.
- **Escalation:** Significant delays or access problems are reported immediately in WhatsApp and recorded in the relevant issue. Scope-changing concerns are handled under Section 8.
- **Course communication:** Questions affecting project scope or official delivery are raised with the instructor or teaching assistant through approved course channels when appropriate.

8. Change Management Plan

1. Propose any material change in scope, requirements, ownership, technology or schedule through a GitHub Issue. State the reason and proposed alternative.
2. Identify affected FR/NFR items, dependent modules, tests, deadlines and person-hour estimates.
3. Discuss the impact with the whole team (WhatsApp and the Issue). **Obtain group agreement before applying a material change**; routine implementation details within an approved task can be handled by its owner and reviewer.
4. If the change alters a major commitment in the submitted Concept Notes or a course deliverable, consult the instructor/TA when appropriate rather than silently redefining the project.
5. Record the decision, update the relevant plans/requirements and test criteria, then implement via a reviewed pull request.
6. Reassess risk and actual-vs-estimated effort during the next weekly review.

Paid-service activation requires explicit group approval and an amended budget; the current project cash budget is zero.

9. Budget Plan

The working project budget is **70 person-hours** and a planned **cash expenditure of 0 TL**. The schedule assumes existing student computers, open-source tooling, local execution and an available free Gemini API allowance. No public hosting or paid LLM access is planned or required for the classroom-oriented demonstration.

Budget item	Planned allocation	Control / assumption
Team effort	10 h analysis, 12 h design, 36 h core development, 12 h tests/delivery	**70 person-hours** in total; **17.5 h per member** as a preliminary equal target
Development tools	0 TL	Use GitHub and existing free/open-source tooling
LLM inference	0 TL	Gemini free allowance only if accessible; mock test responses when offline; no paid fallback
Hosting and infrastructure	0 TL	Run local Docker containers and demonstrate without publishing a production service
Campus data	0 TL	Use authorized data if available, otherwise clearly labeled synthetic datasets

If the free Gemini allowance becomes unavailable, the group will record the reduced live demonstration capability instead of incurring an unapproved cost. Actual effort will be tracked separately from estimates; this plan does not imply that all listed implementation work is guaranteed within 70 hours.

10. Effort Estimation

10.1 Method

The initial estimate uses **expert judgment and analogy with small software-project tasks**, the simple approaches permitted by the assignment. The team allocation sheet provides a provisional top-down baseline of 10 + 12 + 36 + 12 = 70 person-hours. For consistency within Phase 1, each of the four contributors is allocated **2.5 estimated person-hours** for Assignment 1 work (10 person-hours total). Subsequent work is estimated per subsystem and rolled up by phase and person. These are *estimates*, not actual logged work or guaranteed limits. An hour contributed by two people counts as two person-hours. Estimates will be revisited as implementation effort becomes clearer.

10.2 Phase-by-person estimate

This is a **proposed distribution** consistent with the group task sheet's 10/12/36/12-hour phase totals and 17.5-hour per-person targets. It is not an already approved technical work assignment; the effort and role allocation are subject to team review.

Team member	Analysis & plan	Design & skeleton	Core development	Test & delivery	Total
Murat Can Işık	2.5	3	9	3	**17.5**
Yunus Emre Biçer	2.5	3	9	3	**17.5**
Arda Çelik	2.5	3	9	3	**17.5**
Ahmet Sefa Efendioğlu	2.5	3	9	3	**17.5**
Phase total	**10**	**12**	**36**	**12**	**70**

10.3 Task-level effort estimate

Phase	Owner	Estimated task/deliverable	Person-hours
Analysis	Arda	Project definition, scope and success criteria	2.5
Analysis	Murat	Functional/non-functional requirements baseline	2.5
Analysis	Yunus	Project plan, timeline, risks and allocations	2.5
Analysis	Sefa	Cross-document review, release and submission checks	2.5
Design	Arda	Connector contracts, cache keys and mock data schema design	3
Design	Murat	MCP server/tool and agent integration design	3
Design	Yunus	UI layouts, widget and intent-router design	3
Design	Sefa	Gateway, authentication/security and integration-test design	3
Development	Arda	Connectors, synthetic-data adapters, Redis caching and stale fallback	9
Development	Murat	MCP registry/dispatcher and multi-tool integration	9
Development	Yunus	Web dashboard, chat and deterministic router	9
Development	Sefa	Authentication gateway, security controls and integration checks	9
Testing	Arda	Connector mock, cache and fallback-path tests	3
Testing	Murat	MCP tool and orchestration tests	3
Testing	Yunus	UI, browser and routing tests	3
Testing	Sefa	End-to-end/authentication/security tests and final release	3
Total			**70**

Consistency rule: Phase 1 is budgeted at 2.5 hours per person, for 10 hours total. The remaining 60 hours are distributed as 3 hours of design, 9 hours of core development, and 3 hours of testing/delivery per person. If an individual document lists a different estimate, the team should reconcile it in the final cross-document quality review. Actual time worked may differ from these estimates.

11. Task Assignment Matrix and Rationale

11.1 Functional requirements by implementation owner

The following matrix maps the **20 functional requirements in Murat's Requirements Document** to the planned implementation owners. Each owner must coordinate with the relevant interfaces and use the requirement's acceptance criteria for verification. These are *proposed team assignments*, not a claim that coding has already taken place.

Requirements Document ID	Responsible lead	Collaboration / review
FR-01: MCP protocol and schema validation	Murat	Arda (tool schemas), Sefa (integration)
FR-02: Streamable HTTP and stdio transports	Murat	Sefa (integration tests)
FR-03: Cafeteria menu service	Arda	Murat (tool registration)
FR-04: Shuttle schedule service	Arda	Murat (tool registration)
FR-05: Library occupancy service	Arda	Murat (tool registration)
FR-06: Course timetable service	Arda	Sefa (authorization), Murat (tool registration)
FR-07: Gateway authentication and token termination	Sefa	Murat (server identity interface)
FR-08: Rate limiting	Sefa	Arda (Redis access)
FR-09: Redis TTL caching	Arda	Murat (server integration)
FR-10: Stale-if-error fallback	Arda	Sefa (fault tests)
FR-11: Isolated circuit breaker	Arda	Sefa (fault tests)
FR-12: Unified connectors and mock modes	Arda	Murat (schema contracts)
FR-13: LLM multi-tool synthesis	Murat	Yunus (chat interface), Arda (tool data)
FR-14: PII scrubbing of LLM-bound prompts	Sefa	Murat (prompt pipeline)
FR-15: Web login/session state	Yunus	Sefa (mock SSO API)
FR-16: Bilingual deterministic intent router	Yunus	Murat (LLM handoff)
FR-17: SSE chat and tool telemetry	Yunus	Murat (agent streaming)
FR-18: Campus dashboard	Yunus	Arda (service responses)
FR-19: Client errors, timeouts and SSE recovery	Yunus	Sefa (failure tests), Murat (stream behavior)
FR-20: PostgreSQL telemetry/audit logging	Murat	Sefa (audit/security checks)

11.2 Quality and delivery ownership

Requirement group	Coordinating lead	Shared verification approach
NFR-01: Tool performance and latency limits	Murat	Sefa runs latency/load tests; Arda validates cache timing
NFR-02: Redis cache hit ratio	Arda	Sefa reviews simulated workload metrics
NFR-03: Failure isolation	Arda	Sefa coordinates injected-failure tests
NFR-04: Stateless scalability and load distribution	Murat	Sefa verifies container/load scenarios

Requirement group	Coordinating lead	Shared verification approach
NFR-05: Token/secret isolation and TLS	Sefa	Murat verifies LLM prompt boundary
NFR-06: Usability evaluation	Yunus	Team conducts student-user checks where feasible
NFR-07: Browser/viewport portability	Yunus	Team assists with cross-browser checks
NFR-08: Test coverage and offline testability	Each module owner	Sefa combines test reports and verifies mock-only execution
NFR-09: Docker Compose local startup	Sefa	All module owners contribute containers/health checks

Testing plan: Each implementation owner checks their module and records results. Sefa coordinates integrated tests, while Murat verifies MCP/Gemini multi-tool queries, Yunus verifies routing/UI behavior, and Arda verifies connectors/cache/fallback. The Requirements Document's acceptance criteria are the target for validation (including its 10 compound queries, 40 routing samples and performance/security checks). At closure, the team produces a requirement-status table distinguishing **completed, partial, not implemented and not verified**, with evidence and known limitations. Unmet *Must* requirements remain unmet; they are not silently relabeled optional.

11.3 Rationale for task distribution

The allocation gives one lead to each of the four major workstreams and assigns specific integration partners. The aim is a roughly balanced **17.5-hour estimated contribution per person**, with separate responsibilities for development and testing; the number of FR identifiers is not used as a measure of effort. Team members are not assumed to be specialists, and AI coding tools may help with implementation, but each person remains responsible for understanding, checking and testing their delivered changes. Work ownership should be reconfirmed by the team and adjusted if task complexity or available time differs from the initial estimates. All members use traceable GitHub commits and peer-reviewed PRs.

12. Document-Specific Task Matrix

The following task matrix details the specific tasks executed by the team members to author, cross-review, and finalize this Project Plan Document for Assignment 1:

Task Description	Assigned Person	Review / Verification	Effort (h)	Status
Draft scope, organization, objectives, timeline, Gantt chart, and communications	Yunus Emre Biçer	Ahmet Sefa Efendioğlu	0.8	Completed
Draft resource planning, budget, risk register, and change management procedures	Yunus Emre Biçer	Arda Çelik	0.9	Completed
Align technical task allocation with FR-01–FR-20 and NFR-01–NFR-09	Yunus Emre Biçer	Murat Can Işık	0.5	Completed

Task Description	Assigned Person	Review / Verification	Effort (h)	Status
Audit completeness, contribution attribution, and cross-document consistency	Ahmet Sefa Efendioğlu	Yunus Emre Biçer	0.4	Completed
Format document, verify PDF conversion pipeline, and coordinate submission	Ahmet Sefa Efendioğlu	Entire Team	0.4	Completed
Total Effort	**Collaborative Authoring & Review**		**3.0**	**Completed**

Note on Effort Accounting: Yunus Emre Biçer contributed 2.2 hours of direct document authoring, and Ahmet Sefa Efendioğlu contributed 0.8 hours of formal QA cross-review, audit, and release coordination. These hours are accounted for within the 10.0-person-hour "Analysis & Plan" milestone (19 Oct – 1 Nov 2026) defined in Section 10.2.

Planning basis: BIL 481 Project Assignment 1 (19 October 2026); Group-ASYM Concept Notes; Group-ASYM task-allocation sheet; Murat Can Işık's Requirements Document (FR-01–FR-20 and NFR-01–NFR-09). Dates beyond Assignment 1 are internal targets unless subsequently confirmed by course staff.